

F1 Insight

Formula 1 Performance Analytics

Item	Details
Project type	Full-stack data warehousing, OLAP and data mining application
Domain	Formula 1 motorsport analytics
Data source	Jolpica F1 API (Ergast-compatible)
Core stack	React + Vite, FastAPI, MySQL, Scikit-learn

1. Introduction

F1 Insight is a full-stack application for analysing Formula 1 performance using real historical race data. It brings together data collection, an ETL pipeline, a MySQL star-schema data warehouse, OLAP operations, and machine-learning based data mining behind an interactive web dashboard styled after Formula 1 racing, with a black, racing-red, white and charcoal palette.

Formula 1 produces a large and well-structured body of data: results, qualifying sessions, pit stops, circuits, drivers and constructors across more than seven decades. F1 Insight turns this raw data into a dimensional model that can be queried quickly, explored interactively and mined for patterns such as driver performance groups and likely race winners.

1.1 Objectives

- Collect real historical F1 data and load at least 10,000 records into a MySQL data warehouse.
- Design an optimised star schema with fact and dimension tables.
- Provide analytics for drivers, constructors, circuits, races, qualifying and pit stops.
- Support OLAP operations: roll-up, drill-down, slice, dice and pivot.
- Apply data mining: K-Means clustering, race-winner prediction and Apriori association rules.
- Deliver a responsive, motorsport-themed dashboard for desktop, tablet and mobile.

1.2 Core Features

Feature	Description
Data Collection	Real historical data from the Jolpica F1 API (Ergast-compatible).
ETL Pipeline	Extract, clean, transform and load 10,000+ records into MySQL.
Star Schema	Dimensional model with fact and dimension tables.
Analytics	Driver, constructor, circuit, race, qualifying and pit-stop analytics.
OLAP	Roll-up, drill-down, slice, dice and pivot operations.
Data Mining	K-Means, race-winner prediction, Apriori association rules.
Dashboards	Charts, filters and dynamically generated insights.
Modern UI	Timing-table style views, telemetry graphics and driver cards.

2. Technology Stack

The system is split into a React front end, a Python API layer, a MySQL data warehouse and a machine-learning module. Each layer uses widely adopted, well-documented tools.

Layer	Technology	Role in the project
Frontend	React + Vite	Component-based UI with fast development builds.
	JavaScript	Core application language.
	Plain CSS	Custom F1-inspired racing theme (black, racing red, white, charcoal).
	Recharts	Interactive charts and data visualisation.
Backend	Python + FastAPI	Async REST API with Pydantic validation and automatic docs.
	SQLAlchemy + Alembic	ORM models and database migrations.
	Pandas, NumPy	Data transformation and numerical computing.
Database	MySQL 8.0+	Relational data warehouse (Docker setup provided).
Machine Learning	Scikit-learn	K-Means clustering, classification and prediction.
	MLxtend	Apriori association rule mining.

2.1 Why this combination?

- **FastAPI:** FastAPI gives high-performance async handling and request validation through Pydantic, with interactive API documentation generated automatically at /docs.
- **Alembic:** Alembic makes the warehouse schema reproducible; anyone can recreate it with one migration command.
- **React + Vite + Recharts:** Vite and Recharts keep the dashboard responsive and make interactive charting simple.
- **Pandas + Scikit-learn:** Pandas handles cleaning and enrichment during ETL, and the same data frames feed Scikit-learn and MLxtend for mining.

2.2 Prerequisites

Python 3.9+, Node.js 18+ with npm, MySQL 8.0+ (or Docker) and Git.

3. System Architecture

Data flows from the external API through the ETL pipeline into the MySQL warehouse. The FastAPI backend exposes analytics, OLAP and mining services over REST, and the React front end consumes these endpoints to render dashboards.

Stage	Flow
1. Source	Jolpica F1 API (Ergast-compatible historical data)
2. ETL	Extract (rate limiting, retries, disk cache) → Transform (clean, normalise, enrich) → Load (idempotent upserts)
3. Warehouse	MySQL star schema: fact and dimension tables

Stage	Flow
4. Services	FastAPI routes backed by analytics and mining service layers
5. Presentation	React dashboards with Recharts visualisations

3.1 Project Structure

The repository is organised into clearly separated modules:

Directory	Contents
backend/app/analytics	Analytics service layer
backend/app/api	API route handlers
backend/app/mining	ML and data mining modules
backend/app/models, schemas	SQLAlchemy ORM models and Pydantic schemas
backend/etl	ETL pipeline (extract, transform, load)
backend/alembic, tests	Migrations and automated tests
frontend/src	Components, pages, API services, styles and utilities
database, docs, notebooks	Schema scripts, documentation, exploratory Jupyter notebooks
docker-compose.yml	MySQL setup via Docker

4. ETL Pipeline and Data Warehouse

4.1 Extract

Data is fetched from the Jolpica F1 API with rate limiting, automatic retries and disk caching. Caching avoids repeated downloads and makes re-runs fast and polite to the public API.

4.2 Transform

Records are cleaned, normalised and enriched, with data-quality tracking so that issues found during transformation can be reviewed rather than silently dropped.

4.3 Load

Data is loaded with idempotent upserts into the star schema, resolving foreign keys to the dimension tables. Running the pipeline repeatedly will not create duplicates.

4.4 Running the Pipeline

Command	Purpose
python -m etl.run --all	Load all historical data (about 10 to 30 minutes)
python -m etl.run --season 2023	Load a single season for quick testing
python -m etl.run --update	Incremental update for the latest season

4.5 Star Schema

The warehouse uses a star schema: central fact tables hold measurable race events (such as results, qualifying and pit stops), surrounded by dimension tables describing who, where and when (such as drivers, constructors, circuits and seasons). This design keeps aggregate queries simple and fast, and it is

the natural foundation for OLAP. A full description of every table, column, key and derived metric is in the project data dictionary.

5. Analytics and OLAP Operations

The analytics layer covers drivers, constructors, circuits, races, qualifying and pit stops. On top of the warehouse, the OLAP module provides the five classic cube operations:

Operation	Meaning in F1 Insight
Roll-up	Aggregate by climbing a hierarchy, e.g. driver → constructor → season.
Drill-down	Move to finer detail, e.g. season → race → driver.
Slice	Filter the cube on a single dimension.
Dice	Filter on several dimensions at once.
Pivot	Cross-tabulate measures across dimensions.

6. Data Mining and Machine Learning

6.1 K-Means Clustering

Drivers are grouped by performance characteristics. The number of clusters K is selected automatically using the Elbow method together with Silhouette scores, so groupings are data-driven rather than arbitrary.

6.2 Race Winner Prediction

A binary classifier predicts whether a driver will win a race. It is deliberately trained only on pre-race features (an anti-leakage design), so information that would only be known after the race cannot inflate the apparent accuracy.

6.3 Apriori Association Rules

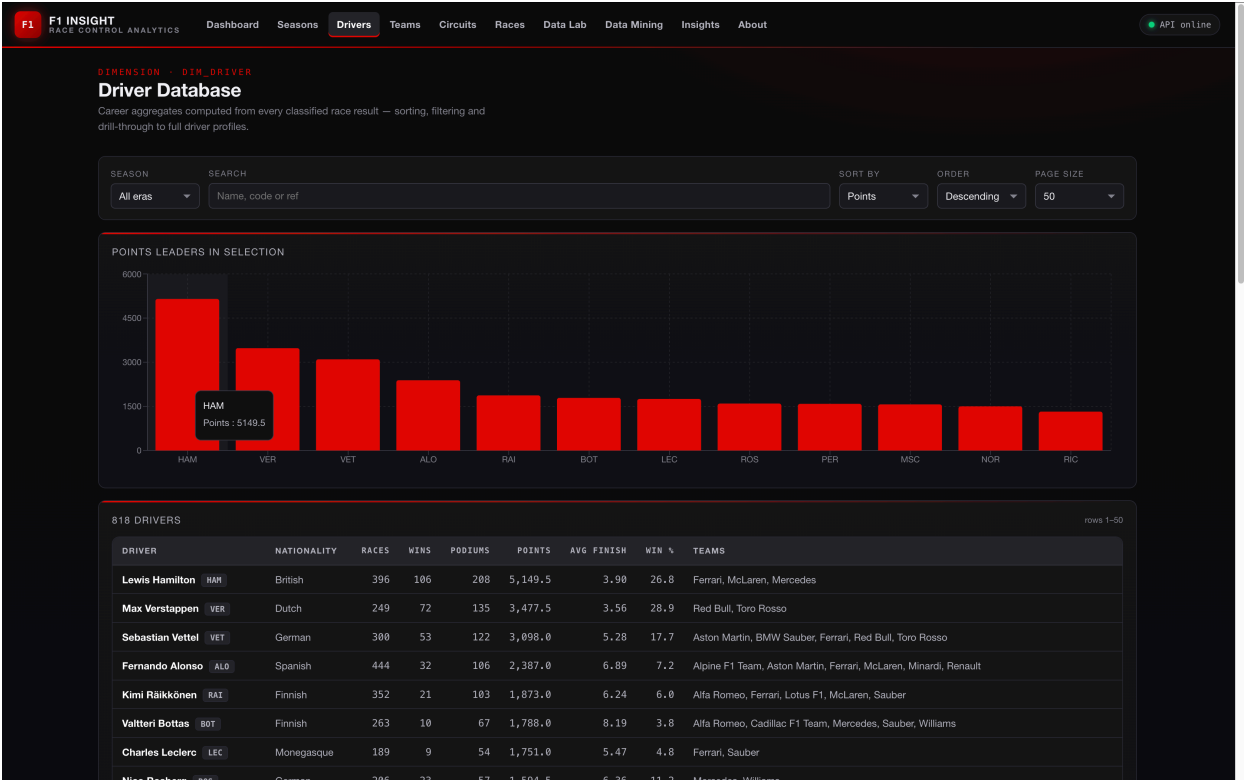
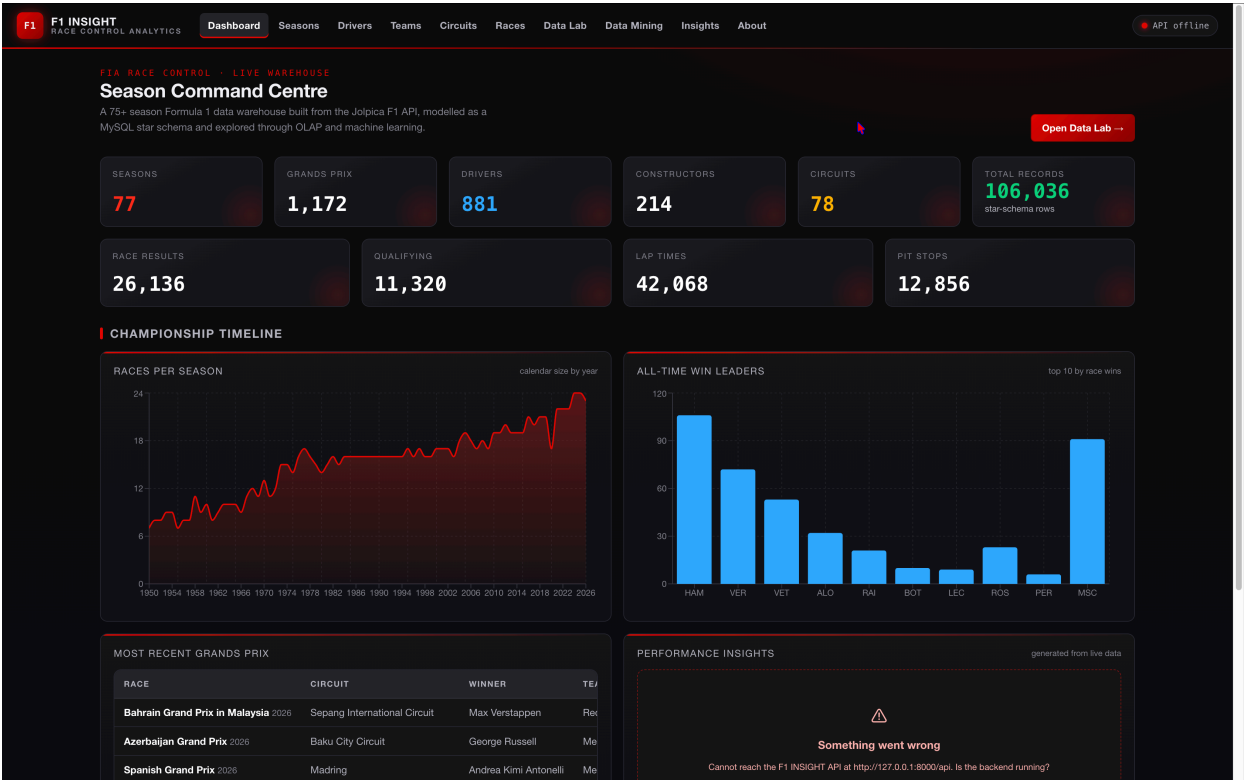
Using MLxtend, the system mines frequent patterns and rules in race outcomes and strategies, for example which combinations of conditions tend to occur together with podium finishes.

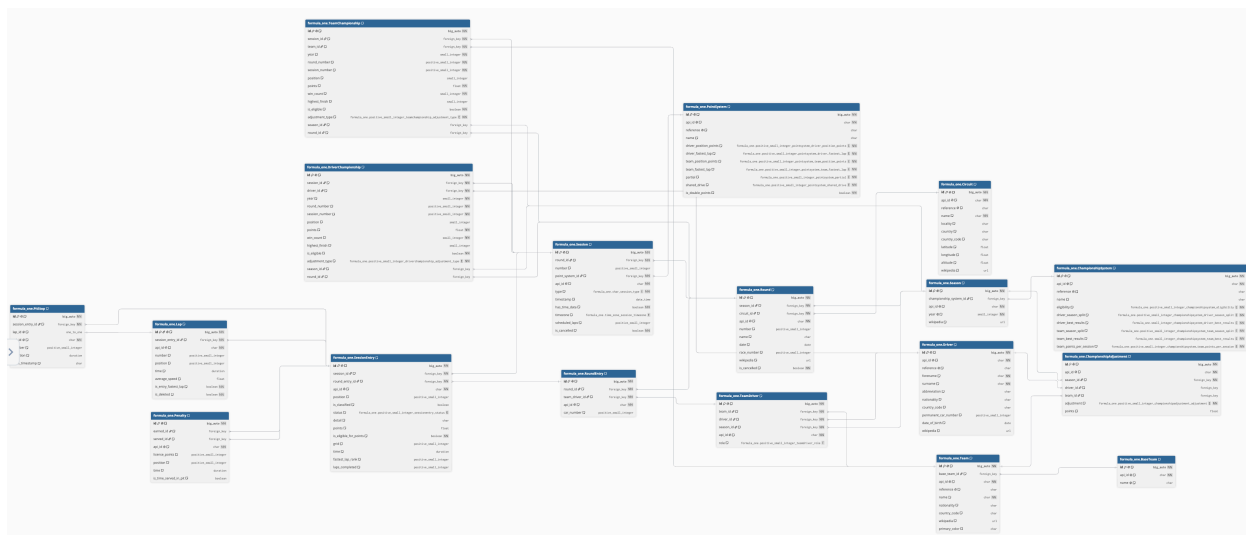
6.4 Performance Classification

A composite scoring system classifies drivers and race results into performance tiers, giving a quick, interpretable summary alongside the mining results.

7. User Interface and Screenshots

The interface follows an F1 broadcast look: motorsport-style timing tables, telemetry-inspired graphics and driver cards. The layout is responsive across desktop, tablet and mobile. Replace the placeholders below with screenshots of your running application (frontend at <http://localhost:5173>).





8. Setup, Testing and Deployment

Step	Action
Environment	Copy .env.example to .env and update database credentials if needed.
Database	docker compose up -d (or create DB f1_insight manually).
Backend	Create a virtual environment, pip install -r requirements.txt, alembic upgrade head.
Data	Run the ETL pipeline (python -m etl.run --all).
API	uvicorn app.main:app --host 127.0.0.1 --port 8000 --reload
Frontend	npm install, then npm run dev (served at localhost:5173).
Testing	pytest tests/ -v from the backend directory.
Production	npm run build for the frontend; uvicorn on 0.0.0.0:8000 for the backend.

Supporting documentation in the docs folder covers the SRS, system architecture, data dictionary, OLAP guide, data mining guide, API reference and setup guide.

9. Conclusion and Future Scope

F1 Insight demonstrates a complete data-driven workflow: collecting real-world data, engineering it into a dimensional warehouse, analysing it with OLAP operations and applying machine-learning techniques, all presented through a polished and responsive interface. The anti-leakage prediction design, idempotent ETL and automatic cluster selection reflect good data-engineering and data-science practice.

Possible future improvements include live race-weekend data, deeper models such as gradient boosting for race prediction, user accounts with saved dashboards, scheduled incremental ETL runs and cloud deployment.

Acknowledgments

Data provided by the Jolpica F1 API. Formula 1 for the inspiration and rich historical data. This project is for educational purposes.